

CLASE N°02

FACULTAD DE INGENIERÍA
EN SISTEMAS, ELECTRONICA
E INDUSTRIAL

fisei

MANEJO DE PARÁMETROS, PASO DE VALORES Y CLASE **CALCULADORA**

PRACTICANDO LA MODULARIDAD Y EL COMPORTAMIENTO DE OBJETOS EN JAVA.

¿Qué se busca aprender?

- Modularidad: organizar el código en clases y métodos.
- Objetos: representar elementos del mundo real mediante atributos y comportamientos.
- Métodos: definir las acciones que puede realizar cada objeto.
- Reutilización: aprovechar el código existente sin tener que escribirlo nuevamente.



PARÁMETROS VS. ARGUMENTOS

OBJETIVO: Explicar la diferencia conceptual y práctica entre ambos términos.

DEFINICION:

- Parámetro: Es la variable declarada en la firma del método. Funciona como un espacio reservado para recibir datos.
- Argumento: Es el valor real que le enviamos al método cuando lo llamamos desde otra parte del código.

EJEMPLO CALCULADORA:

```java
// 'a' y 'b' son PARÁMETROS (Definición del método)
public double sumar(double a, double b) {
    return a + b;
}

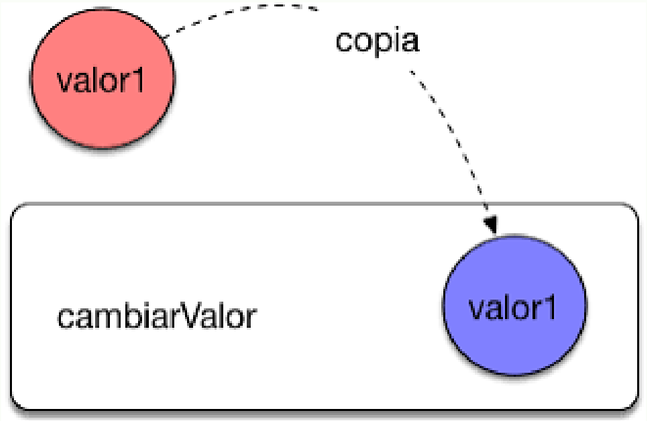
// '15.5' y '4.5' son ARGUMENTOS (Llamada al método)
double resultado = miCalc.sumar(15.5, 4.5);
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

PASO POR VALOR EN JAVA

- Puntos clave:
- En Java, los argumentos SIEMPRE se pasan por valor.
- Cuando le envías una variable a un método, Java entrega una copia de ese valor.
- Modificar la variable del parámetro dentro del método NO afecta la variable original que estaba afuera.

- PREGUNTA:
- “Si declaro int x = 5; y se lo paso a un método que le suma +10 a su parámetro, ¿cuánto vale x fuera del método después de ejecutarlo?”



MÚLTIPLES PARÁMETROS

- Un método puede recibir la cantidad de parámetros que necesite.
- Se separan por comas (,).
- Pueden ser del mismo tipo de dato o de tipos combinados (String, int, double, boolean).

EJEMPLO DE CODIGO:

```java
public void registrarOperacion(String tipoOperacion, double resultado, boolean exitosa) {
    System.out.println("Operación: " + tipoOperacion);
    System.out.println("Resultado: " + resultado);
    System.out.println("¿Exitosa?: " + exitosa);
}
```

CLASE CALCULADORA

- Objetivo: Mostrar la estructura de la clase práctica que van a codificar en vivo en el IDE.
- Código a presentar:

```java
public class Calculadora {

    // Método void sin retorno
    public void mostrarMenu() {
        System.out.println("=== CALCULADORA BÁSICA EN JAVA ===");
        System.out.println("1. Sumar | 2. Restar | 3. Multiplicar | 4. Dividir");
    }

    // Métodos con retorno y parámetros
    public double sumar(double a, double b) {
        return a + b;
    }

    public double restar(double a, double b) {
        return a - b;
    }

    public double multiplicar(double a, double b) {
        return a * b;
    }

    // Validación especial: División entre cero
    public double dividir(double a, double b) {
        if (b == 0) {
            System.out.println("Error: No se puede dividir entre cero.");
            return 0;
        }
        return a / b;
    }
}
```

EJECUCIÓN EN VIVO (CLASE MAIN)

- Contenido: Cómo instanciar la clase Calculadora y probar cada uno de sus métodos en la clase principal.
- Código a presentar:

```java
public class Main {
    public static void main(String[] args) {
        // Instanciación del objeto
        Calculadora calc = new Calculadora();

        // Llamada al método void
        calc.mostrarMenu();

        // Invocación de métodos con retorno
        double totalSuma = calc.sumar(10.5, 5.5);
        System.out.println("Resultado Suma: " + totalSuma);

        double totalDiv = calc.dividir(20, 0); // Prueba de la validación
    }
}
```

CONCLUSIONES DE LA SESIÓN



Flexibilidad:

Flexibilidad: Los parámetros permiten que los métodos procesen distintos datos sin reescribir código.



Seguridad

El paso por valor protege las variables originales de ser modificadas accidentalmente.



Validación

Los métodos deben ser capaces de validar sus entradas (como prevenir divisiones para b == 0).



Preparación

Listos para la sesiónde "Completar Código" con la clase Producto


Gracias por su atencion